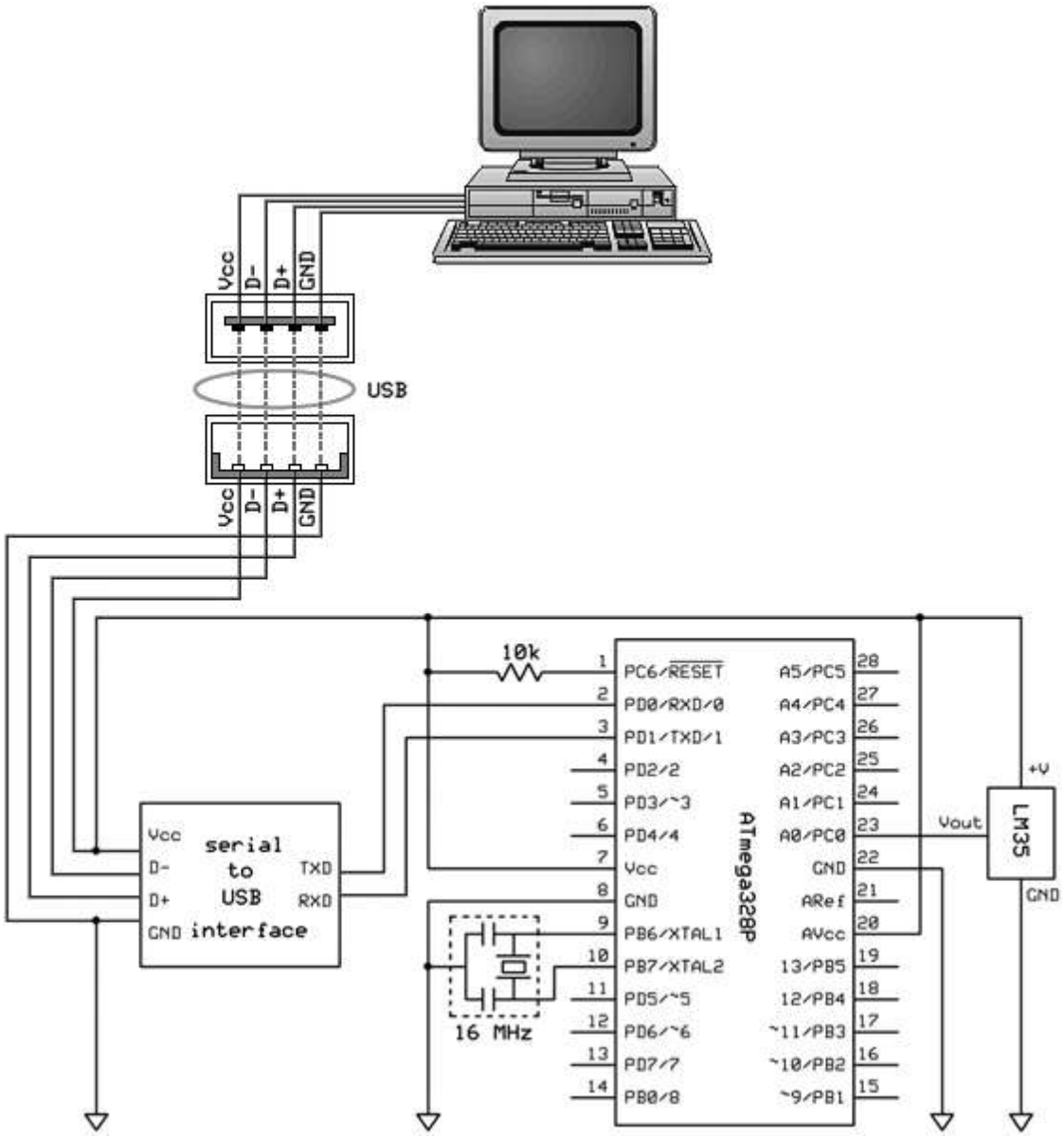
Embedded Systems Development

Exercise 7

The "temp logger" is a system that consists of an ATmega328P microcontroller-based embedded subsystem and a .NET application running on a PC. The embedded subsystem captures ambient temperature using an LM35 temperature sensor and listens to the serial port that connects it to the PC via a serial to USB interface. When it hears ASCII character 'Y' through the serial port, the embedded subsystem starts transmitting the captured temperature data over the serial communication every second and keeps listing to the serial port. When the embedded subsystem hears any other ASCII character through the serial port, it stops transmitting temperature data immediately and keeps listening to the serial port. The schematic diagram of the embedded subsystem is shown below.

Schematic Diagram



1. Write the required firmware for the embedded subsystem using Arduino wrapper functions.

2. The source code of another version of the firmware for the embedded subsystem written in C using the Arduino IDE is shown below. Fill in the blanks in the source code using the C programming language without using any Arduino wrapper functions. Use the labeled constants defined in avr/io.h.

```
1  // tempLoggerC.ino
2
3  #include <avr/io.h>                          // AVR I/O definitions
4  #define F_CPU 16000000UL                     // CPU clock speed
5  #include <util/delay.h>                      // definition of _delay_ms()
6  #define BASE 10                              // base for decimal numbers
7  #include <stdlib.h>                          // definition of itoa()
8
9  void initADC();                              // function prototype for initADC()
10 void initUSART();                             // function prototype for initUSART()
11 char receiveChar();                           // function prototype for receiveChar()
12 void transmitChar(char);                      // function prototype for transmitChar()
13 void printString(char *ch);                   // function prototype for printString()
14
15 int main(void)
16 {
17   char ch = 'N';                             // variable to hold received character
18   char str[4];                               // array to hold character converted integer
19   initADC();                                 // initialize ADC
20   initUSART();                               // initialize USART
21
22   while (1)
23   {
24     if (UCSR0A & (1<<RXC0))                  // if an unread character in data register
25       ch = receiveChar();                    // get character
26     if (ch == 'Y')                           // if character received equals 'Y'
27     {
28       ..................................     // start ADC
29       ..................................     // poll for ADC complete
30       float vIn  = ADCW * 5.0 / 1024.0;      // get analog input voltage
31       int temp = vIn * 100;                  // compute temperature in Celsius
32       itoa(temp, str, BASE);                 // convert integer to ASCII characters
33       printString(str);                      // print temperature
34       printString("\n");                     // print new line character
35       _delay_ms(1000);                       // wait for one second
36     }
37   }
38   return 0;
39 }
40
```

```
41 void initADC()                                        // initialize ADC
42 {
43   DDRC = DDRC & ~(1<<PORTC0);                        // configure PC0 (ADC0) as input
44   ..................................................// set Vref as AVcc (5V)
45   ..................................................// multiplex PC0 as ADC0
46   ..................................................// set ADC clock speed as 125kHz
47   ..................................................// enable ADC
48 }
49
50 void initUSART()                                       // initialize USART
51 {
52   ..................................................// set baud rate as 9600bps
53   ..................................................//
54   ..................................................// specify char size as 8-bit
55   ..................................................// enable RX and TX
56 }
57
58 char receiveChar()                                     // receive a character
59 {
60   ..................................................// poll for receive complete flag
61   ..................................................// return character in data register
62 }
63
64 void transmitChar(char ch)                             // transmit a character
65 {
66   ..................................................// poll for data register empty flag
67   ..................................................// write character to data register
68 }
69
70 void printString(char *ch)                             // print a string of characters
71 {
72   ..................................................// repeat till '\0' character
73   ..................................................
74   ..................................................// transmit character
75   ..................................................// increment to next character
76   ..................................................
77 }
```

1146 bytes

3. Line 18 of the above source code declares an array to store the character representation of the temperature in integer. If the measurement range of the LM35 temperature sensor is −55 °C to 150 °C, explain why this array requires at least four elements.

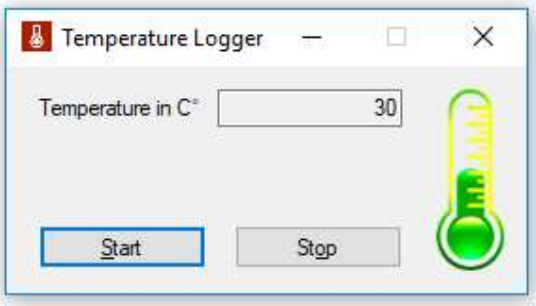
4. In the C programming language, a stream is a sequence of bytes coming from and/or going to a device such as a keyboard, monitor, disk, or port. A stream provides a logical interface for accessing different devices, making I/O device independent. A sequence of bytes flowing into a program is called an input stream, while a sequence of bytes flowing out of a program is called an output stream. The keyboard is normally associated with the standard input stream and the monitor is normally associated with the standard output stream.

   The FDEV_SETUP_STREAM() macro defined in stdio.h can be used to configure a user-defined stream for a device such as a serial port. Once the configured stream is assigned to the standard input and output streams, standard C language I/O functions such as getchar(), putchar(), scanf(), and printf() defined in stdio.h can be used to perform input and output through the serial port.

   Using Microchip Studio IDE, rewrite the firmware for the embedded subsystem of "temp logger" in C. Set up a stream to access the serial port and use the standard C language I/O functions getchar() and printf() to read characters from and write temperature data to the serial port, respectively. Use the labeled constants defined in avr/io.h.

5. Discuss the pros and cons of using the standard C language I/O functions defined in stdio.h in the context of embedded systems development.

6. Consider the following GUI of the .NET application of "temp logger".



When the .NET application of "temp logger" is executed, it opens a connection to the serial port of the PC to which the embedded subsystem of "temp logger" is connected. When the user clicks the "Start" button, the .NET application sends the ASCII character 'Y' to the embedded subsystem through the serial port and starts displaying the temperature data received from the embedded subsystem on the GUI every second.

When the "Stop" button is clicked, the .NET application sends the ASCII character 'N' to the embedded subsystem through the serial port, immediately stops displaying temperature data, and clears the GUI. It also sends the ASCII character 'N' to the embedded subsystem and then closes the serial port connection when the user terminates the .NET application.

Develop the .NET application of "temp logger" in C#. Provide both the GUI design details and the source code. Use the SerialPort component and handle its DataReceived event. Note that the DataReceived event handler and the GUI run on different threads. Therefore, appropriate cross-thread invocation techniques must be used when updating GUI controls from the DataReceived event handler.

GUI Design Details

Class	Object	Property	Value
Form	TempForm	File Name	Temp.cs
		Name	TempForm

Source Code